Mermaid.js Syntax and Diagram Guide

🧜 Mermaid.js — Complete AI Skill Reference

General Structure

Every Mermaid diagram starts with a **diagram type declaration**. The code is wrapped in a fenced code block using ```` ```mermaid ```` in Markdown.

````
```mermaid
<diagram-type>
    <diagram-content>
````

````
### Init Directive (Frontmatter Config)
Place at the **very first line** to configure theme/layout:
```mermaid
%%{init: {'theme': 'dark', 'logLevel': 'debug'}}%%
graph TD
    A --> B
````

Or use YAML frontmatter (v10.5.0+):

```
---
config:
  theme: forest
  layout: elk
---
flowchart LR
    A --> B
```

---

1. Flowchart / Graph

Direction Keywords

Keyword	Direction
`TD` / `TB`	Top → Bottom
`LR`	Left → Right
`BT`	Bottom → Top
`RL`	Right → Left

Node Shapes

Syntax	Shape
`A[text]`	Rectangle
`A(text)`	Rounded rectangle
`A([text])`	Stadium / pill
`A{text}`	Diamond (decision)
`A{{text}}`	Hexagon
`A((text))`	Circle
`A[/text/]`	Parallelogram (lean right)
`A[\text\]`	Parallelogram (lean left)
`A[/text\]`	Trapezoid
`A[\text/]`	Inverted trapezoid
`A>text]`	Asymmetric / flag

Syntax	Shape
`A[(text)]`	Cylinder / database
`A@{ shape: rect }`	New v11.3+ shape syntax

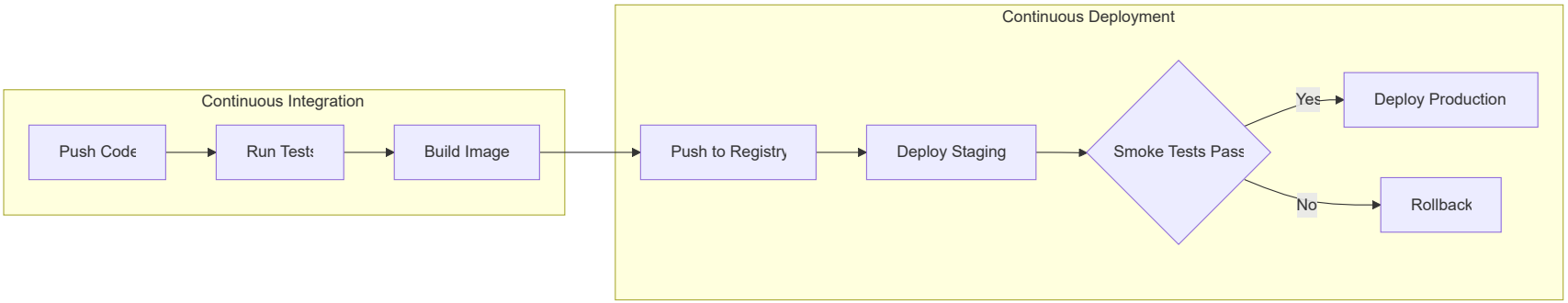
v11.3+ Extended Shapes (selected)

Short Name	Description
`diam`	Diamond
`cyl` / `db`	Cylinder/database
`doc`	Document
`docs`	Multi-document
`hex`	Hexagon
`stadium` / `pill`	Stadium
`fr-rect` / `subprocess`	Framed rectangle
`bolt`	Lightning bolt / com-link
`cloud`	Cloud
`hourglass`	Collate
`flag`	Paper tape
`cross-circ`	Summary

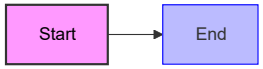
Arrow / Edge Types

Syntax	Description
`-->`	Solid arrow
`-.->`	Dotted arrow
`==>`	Thick arrow
`---`	Solid line (no arrow)
`-.-`	Dotted line (no arrow)
`===`	Thick line (no arrow)
`` `--> ``	label
`-- text -->`	Alternative label syntax
`--o`	Circle at end
`--x`	Cross at end
`<-->`	Bidirectional

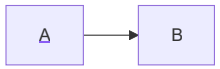
Subgraphs



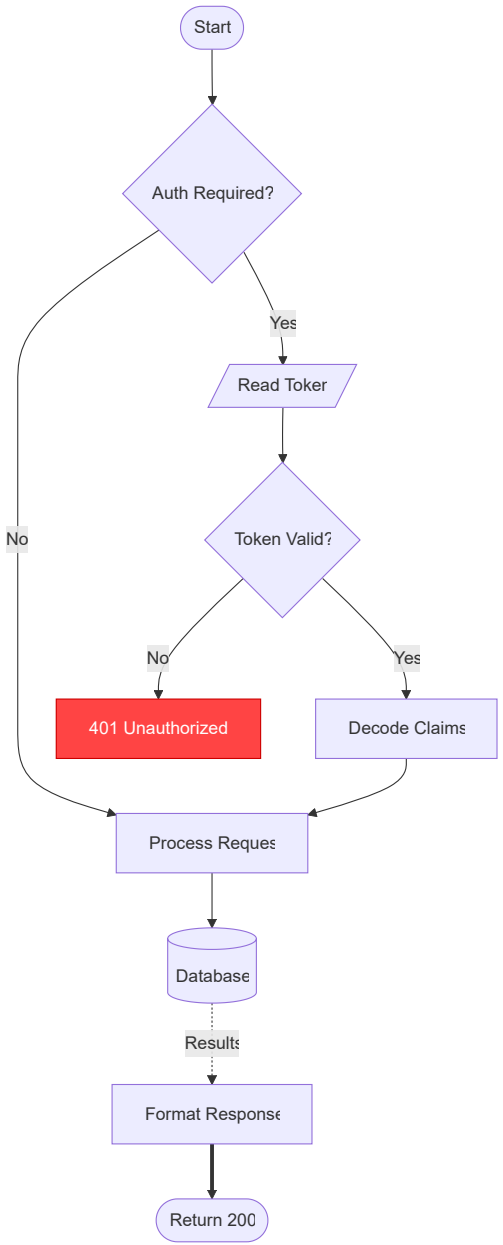
Styling



Click Interactions

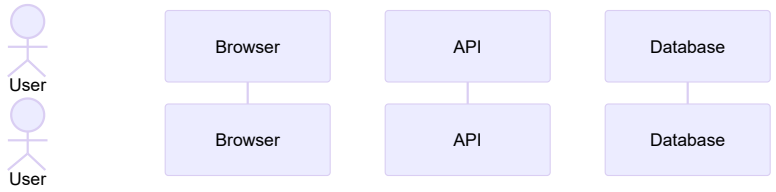


Complex Flowchart Example



2. Sequence Diagram

Participants & Actors

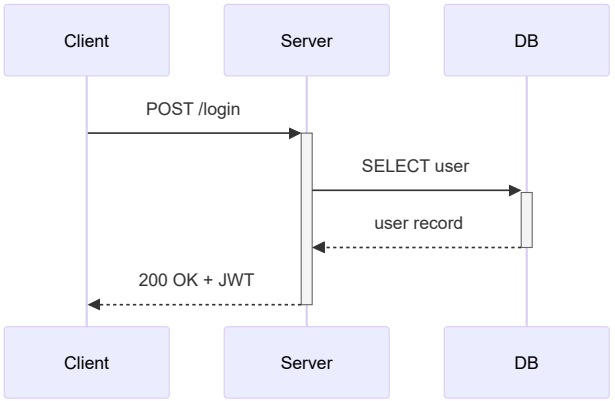


- → stick figure
- → box
- → alias/display name

Message Arrow Types

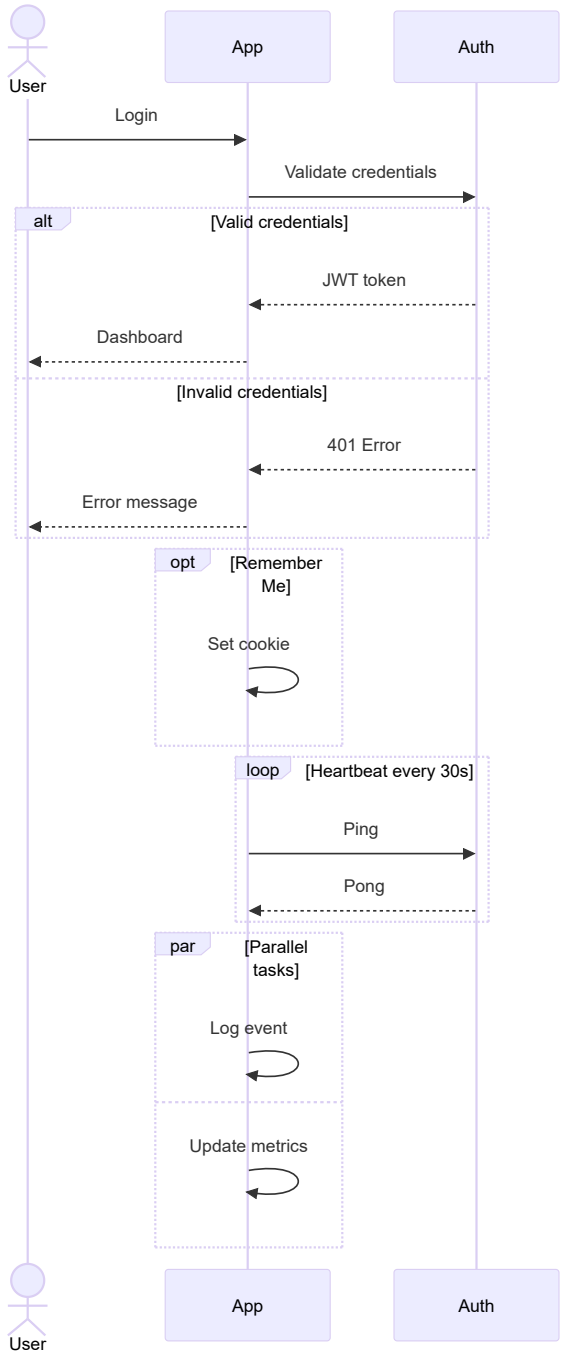
Syntax	Description
`->`	Solid line, no arrowhead
`-->`	Dotted line, no arrowhead
`->>`	Solid line with arrowhead
`-->>`	Dotted line with arrowhead
`<<->>`	Bidirectional solid (v11.0.0+)
`<<-->>`	Bidirectional dotted (v11.0.0+)
`-x`	Solid with cross (lost message)
`--x`	Dotted with cross
`-)`	Solid open arrow (async)
`--)`	Dotted open arrow (async)

Activation Bars

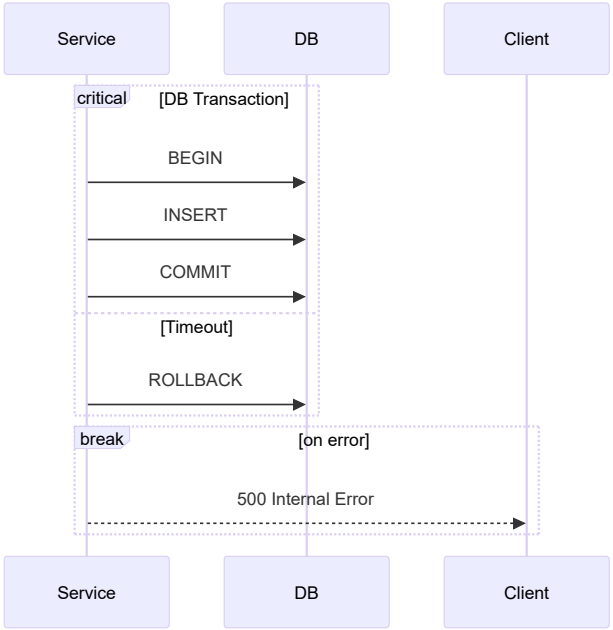


`+` activates, `-` deactivates.

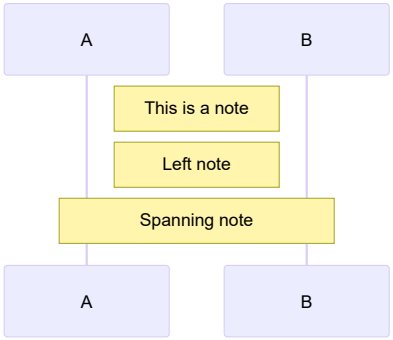
Control Flow Blocks



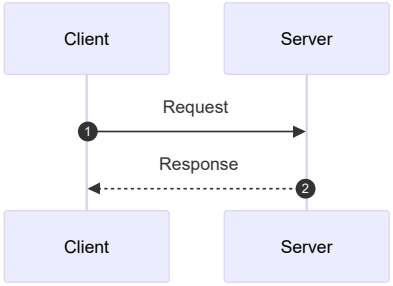
Critical & Break



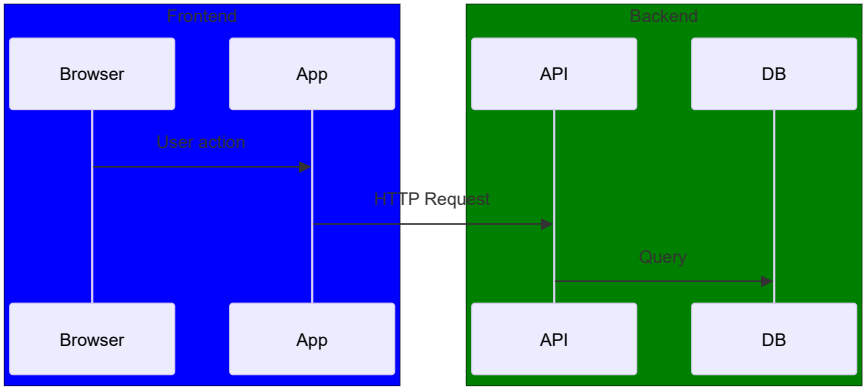
Notes



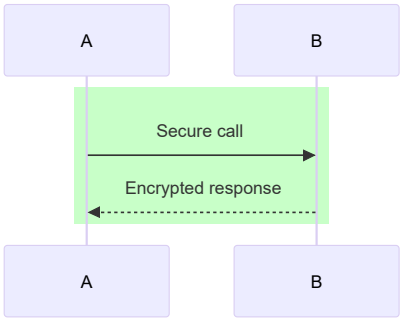
Autonumber



Actor Groups / Boxes



Background Highlighting



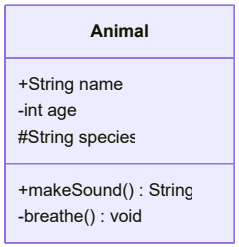
---

3. Class Diagram

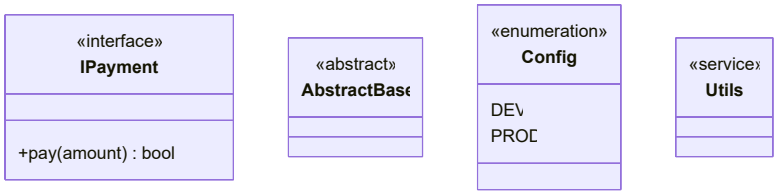
Visibility Markers

Marker	Meaning
`+`	Public
`-`	Private
`#`	Protected
`~`	Package / internal

Class Syntax



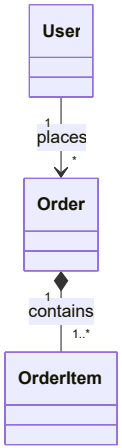
Annotations



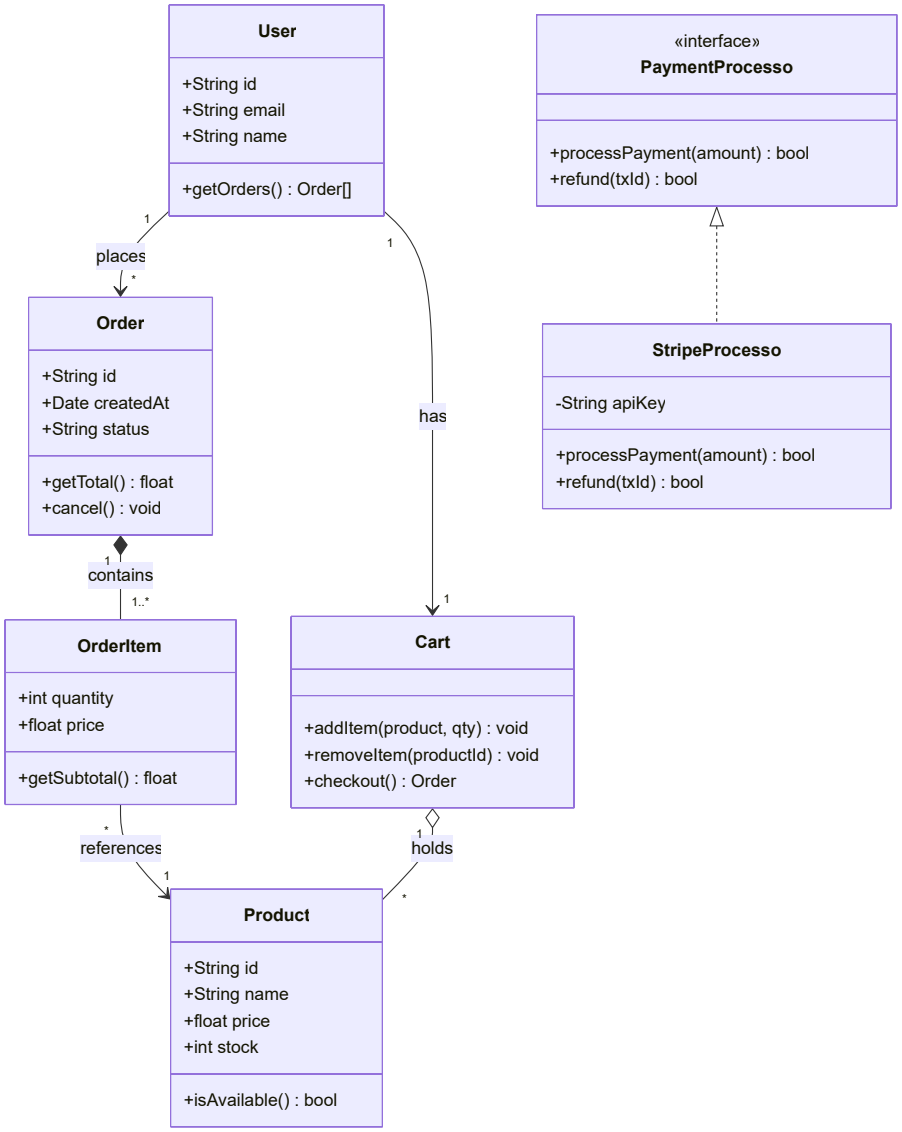
Relationships

Syntax	Relationship
`A <|-- B`	B inherits A (extends)
`A *-- B`	Composition (A owns B)
`A o-- B`	Aggregation (A has B)
`A --> B`	Association (uses)
`A ..> B`	Dependency
`A ..|> B`	Implementation (B implements A)
`A -- B`	Link (solid)
`A .. B`	Link (dotted)

Cardinality Labels



Full E-Commerce Class Diagram Example



---

4. Entity-Relationship Diagram

Attribute Markers

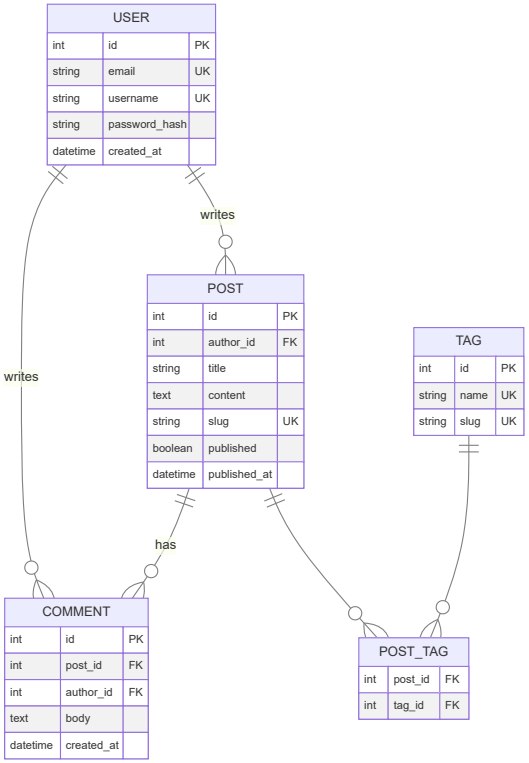
Marker	Meaning
PK	Primary Key
FK	Foreign Key
UK	Unique Constraint

Cardinality Notation

Syntax	Meaning
||--||	One to one

Syntax	Meaning
`||--o{`	One to zero-or-many
`||--|{`	One to one-or-many
`}o--o{`	Zero-or-many to zero-or-many
`|`	Exactly one
`o`	Zero
`{`	Many

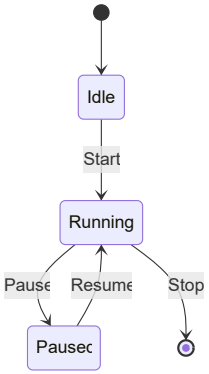
Full Blog Schema Example



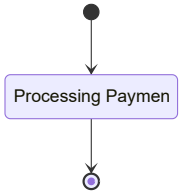
---

5. State Diagram (v2)

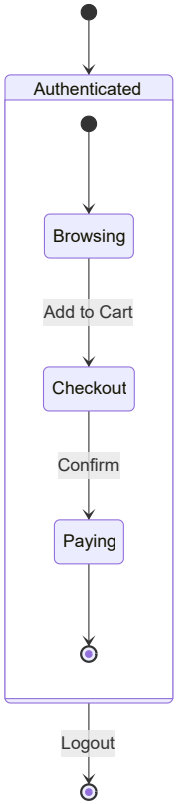
Basic Syntax



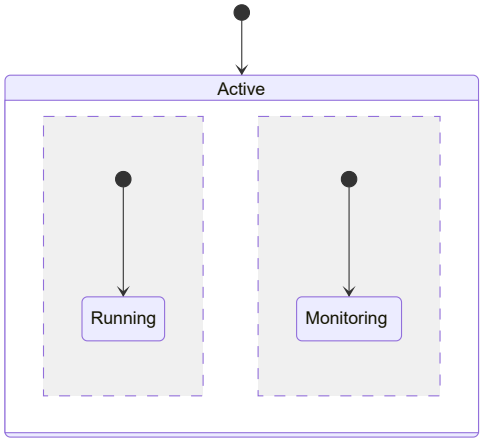
State with Description



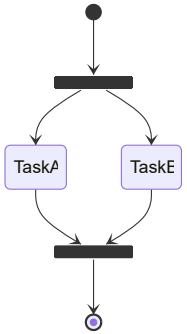
Nested / Composite States



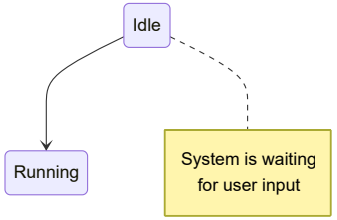
Concurrency (Parallel States)



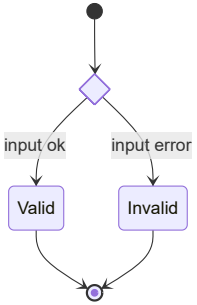
Fork & Join



Notes in State Diagrams



Choice (Conditional)



6. Gantt Chart

Task Syntax

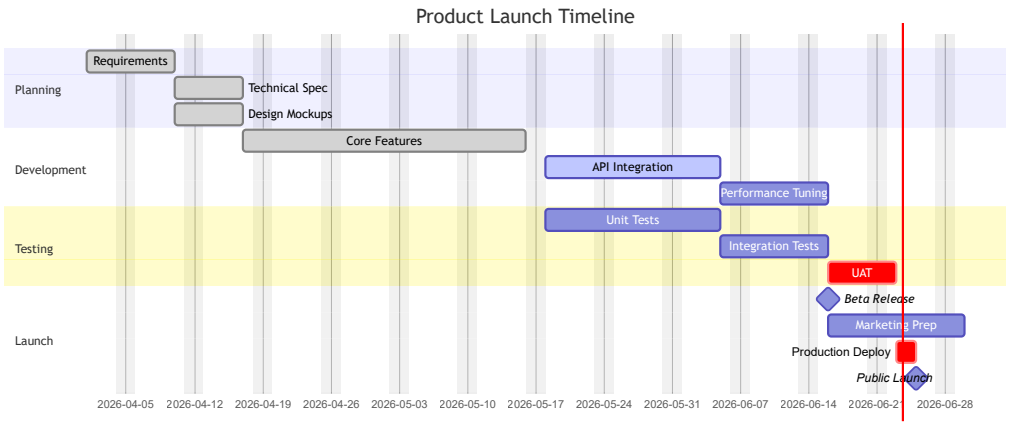
```
<task name> :<status,> <id,> <start>, <duration>
```

- Start: absolute or relative
- Duration: (days), (weeks), (hours)

Task Status Keywords

Keyword	Effect
`done`	Completed (filled)
`active`	In progress (highlighted)
`crit`	Critical path (red)
`milestone`	Zero-duration marker

Full Project Timeline Example



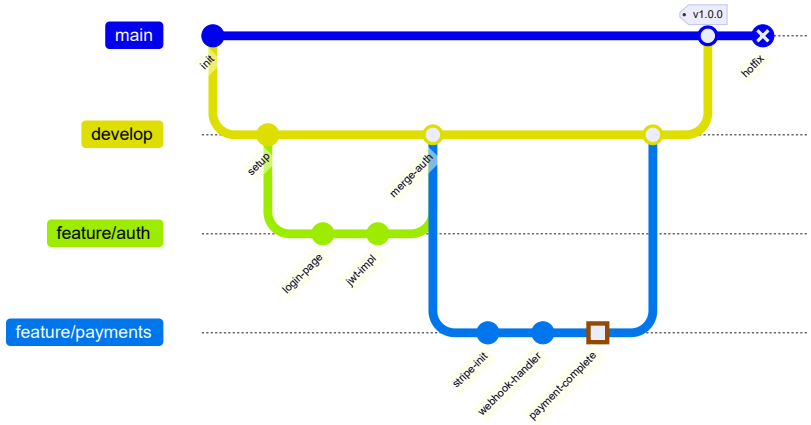
---

7. Git Graph

Core Commands

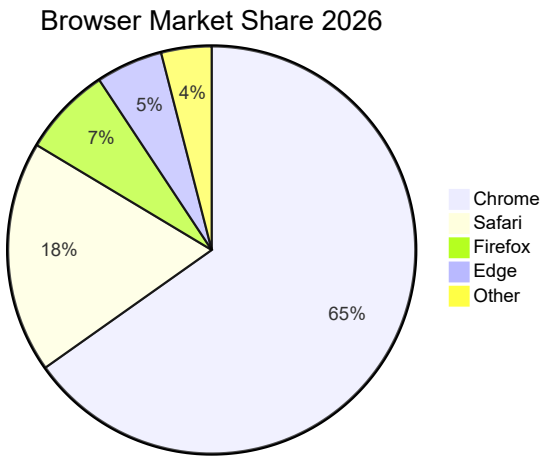
Command	Description
`commit`	Create a commit on current branch
`commit id:"name"`	Commit with custom label
`commit type: HIGHLIGHT`	Highlighted commit
`commit type: REVERSE`	Reversed/reverted commit
`commit type: NORMAL`	Standard commit (default)
`commit tag:"v1.0"`	Add tag to commit
`branch <name>`	Create a new branch
`checkout <name>`	Switch to branch
`merge <name>`	Merge branch into current
`cherry-pick id:"id"`	Cherry-pick a commit

Full Git Flow Example

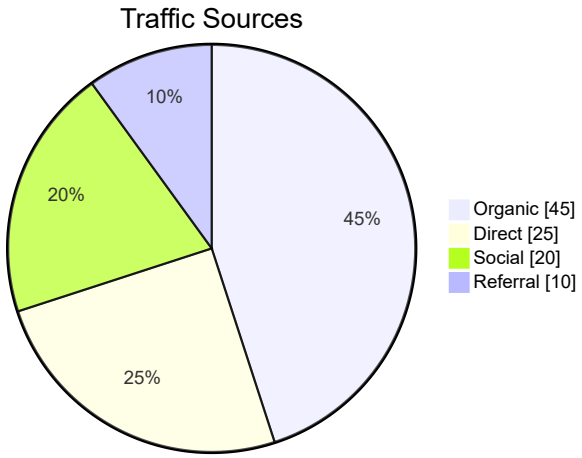


---

8. Pie Chart

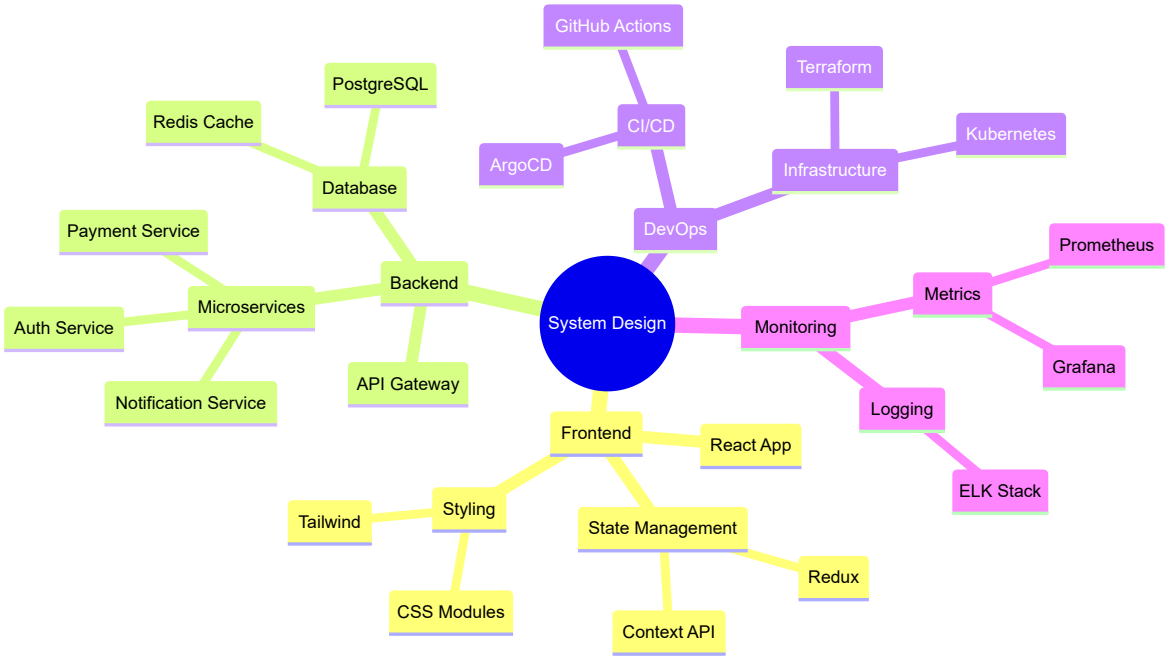


Use `showData` to display values:



9. Mind Map

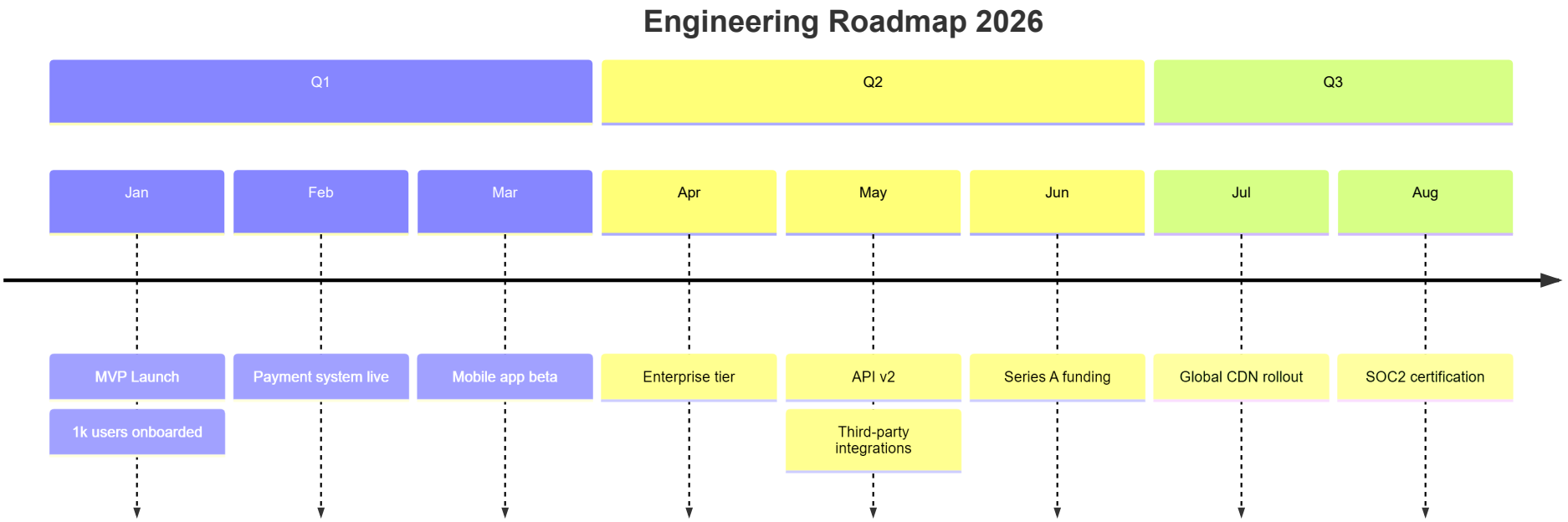
Indentation defines hierarchy. Root uses `((double parens))` for circle.



Node shapes work the same as flowchart: `((circle))`, `[rect]`, `(rounded)`, `{{hexagon}}`

---

10. Timeline



---

11. C4 Diagram

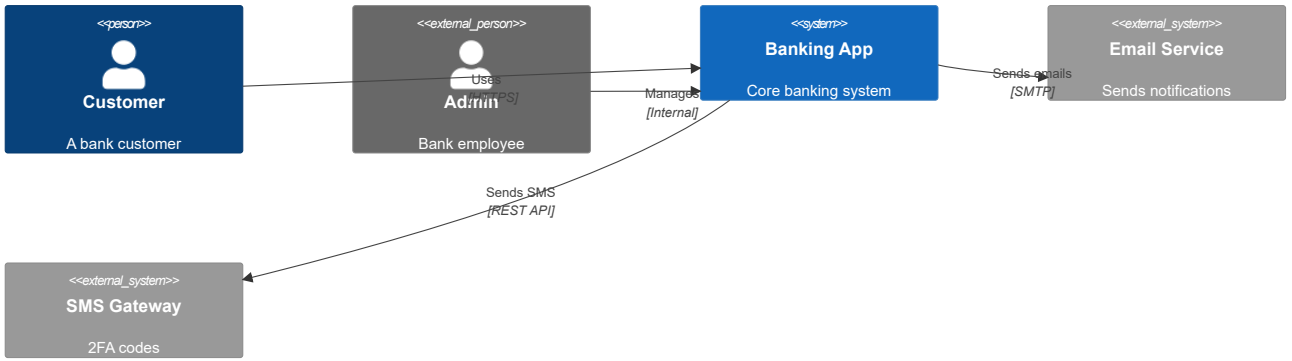
Types

- — System context
- — Container level
- — Component level
- — Dynamic/sequence view

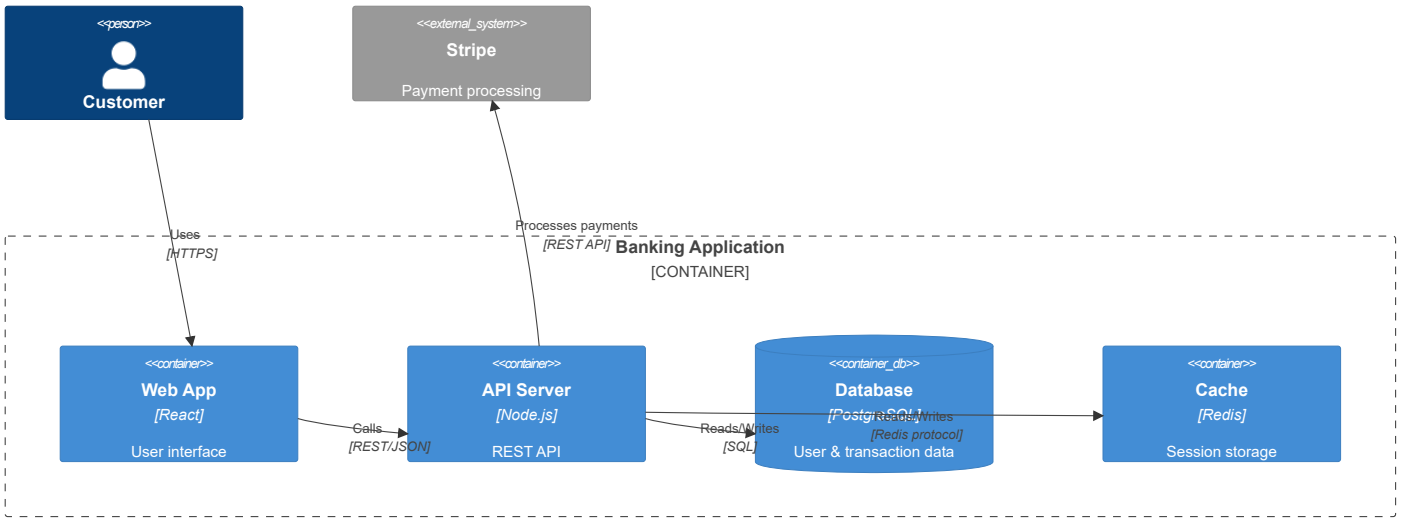
Elements

Element	Syntax
Person	`Person(alias, "label", "desc")`
External Person	`Person_Ext(alias, "label", "desc")`
System	`System(alias, "label", "desc")`
External System	`System_Ext(alias, "label", "desc")`
Container	`Container(alias, "label", "tech", "desc")`
Container DB	`ContainerDb(alias, "label", "tech", "desc")`
Component	`Component(alias, "label", "tech", "desc")`
Boundary	`System_Boundary(alias, "label") { ... }`
Relationship	`Rel(from, to, "label", "tech")`

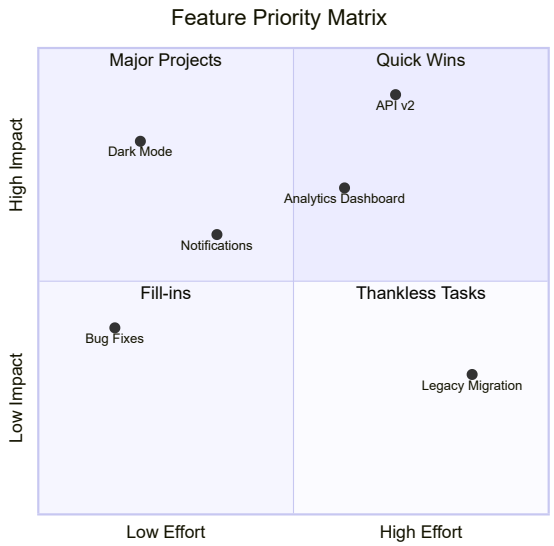
System Context — Banking App



Container View — Banking App

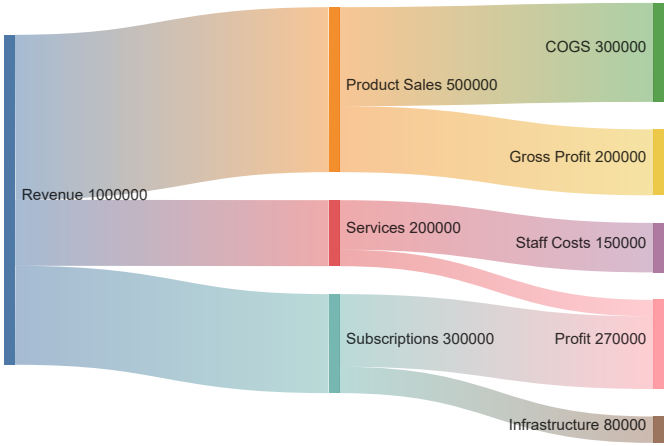


12. Quadrant Chart

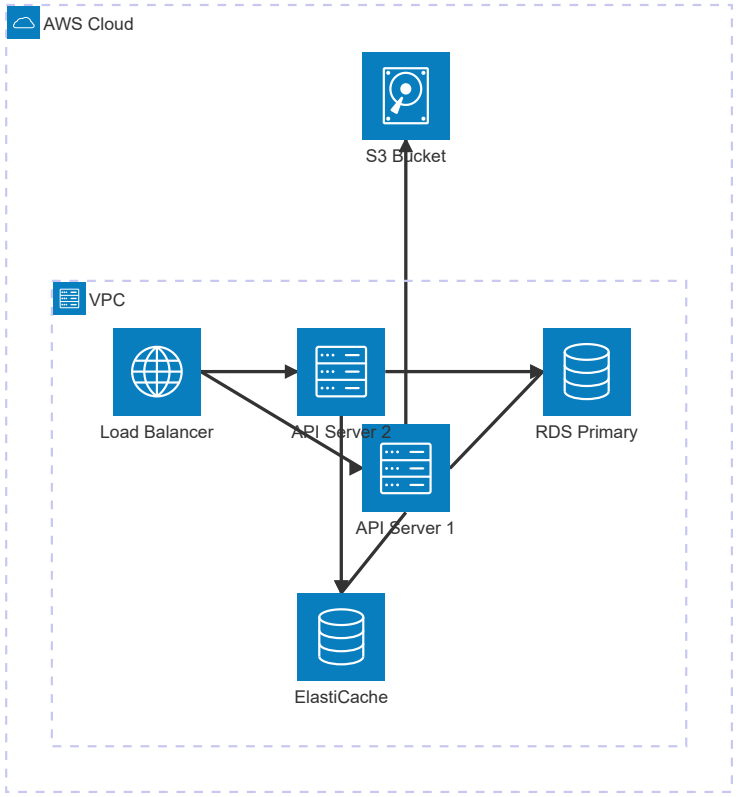


13. Sankey Diagram

CSV format: `Source,Target,Value`



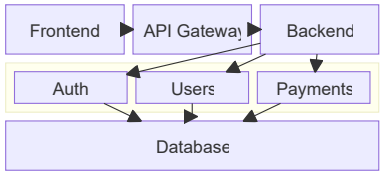
14. Architecture Diagram (beta)



Service icons: `server`, `database`, `disk`, `internet`, `cloud`
Arrow syntax: `service1:Side --> Side:service2` (sides: `L`, `R`, `T`, `B`)

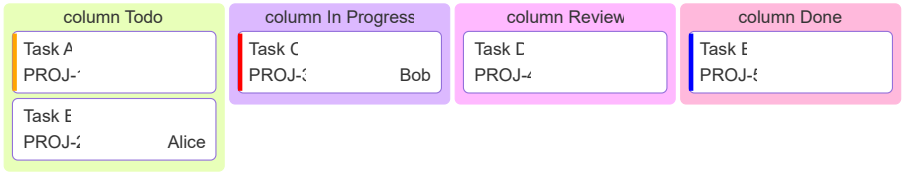
---

15. Block Diagram



---

16. Kanban



---

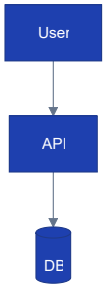
17. Theming

Built-in Themes

Theme	Description
`default`	Light, blue/purple
`dark`	Dark background
`forest`	Green palette
`neutral`	Grayscale
`base`	Bare — for full customization

Apply Theme

Full Custom Theme



---

18. Common Syntax Errors & Rules

Critical Rules

- Node IDs **cannot contain spaces** — use or
- Node IDs must be **unique** within a diagram
- No **semicolons** in class diagram attributes
- `sequenceDiagram` uses not for arrowhead messages
- `stateDiagram-v2` (use v2, not v1)
- `gitGraph` not (capital G)
- Subgraph IDs must be unique; direction can be set:
- Special characters in node labels: wrap in quotes
- Markdown strings (bold/italic in labels): use **bold** text`"]`

Arrow Syntax Quick Checks

Diagram	Correct	Wrong
Sequence request	`->>`	`->`
Sequence response	`-->>`	`-->`
Flowchart	`-->`	`->`
State transition	`-->`	`->`
Class relationship	`<|--`	`<--`

---

19. Quick Cheat Sheet

```
flowchart TD          sequenceDiagram       classDiagram
erDiagram             stateDiagram-v2       gantt
gitGraph              pie                   mindmap
timeline              C4Context             C4Container
C4Component           C4Dynamic             quadrantChart
sankey-beta           architecture-beta     block-beta
kanban
```

Universal Init Config Template

```
%%{init: {
  'theme': 'default',
  'fontFamily': 'monospace',
  'flowchart': { 'curve': 'basis', 'padding': 20 },
  'sequence': { 'actorMargin': 50, 'showSequenceNumbers': true },
  'gantt': { 'axisFormat': '%m/%d' }
}}%%
```

> **Live Editor**: mermaid.live — paste any diagram to preview instantly.
> **Official Docs**: mermaid.js.org
> **Version**: This reference covers Mermaid v11.x

How to Use This as an AI Skill

Copy the entire reference above into a `SKILL.md` file (or your AI platform's skill/system prompt). When triggering the skill, the model should:

1. **Identify the diagram type** from the user's description
2. **Select the correct declaration keyword** (, , etc.)
3. **Apply valid syntax** using only the patterns in this reference
4. **Wrap output** in fenced code blocks
5. **Validate** against the common errors table before responding

---